

II PROJECT PLANNING

Project Planning

- Project planning means **dividing the overall work** into smaller tasks, assigning them to team members, and **thinking ahead** about possible problems and how to solve them.
- A **project plan** is created at the **beginning** of a project. It:
 - Explains how the work will be done.
 - Is shared with the **team and customers**.
 - Helps in **tracking the progress** of the project.

Stages of Planning

Project planning happens at **three main points** during a software project:

1. **Proposal Stage**
 - When you're trying to **win a contract** (bidding for a project).
2. **Project Startup**
 - After the contract is won and the project is about to begin.
3. **Ongoing (Throughout the Project)**
 - As the project moves forward, the plan is updated based on experience and progress.

1. Proposal Planning

- At this early stage, you may only have **basic or outline requirements**.
- The goal is to gather enough information to:
 - Decide how much to **charge the customer** (project pricing).
 - Estimate the **total cost** of the software, including:
 - **Staff** costs
 - **Hardware and software** costs

- Other development expenses

2. Project Startup Planning

- At this stage, you understand the **requirements better**, but you don't have full details of the design or coding yet.
- The goal is to:
 - Create a plan detailed enough to **set the budget** and **assign team members**.
 - Allocate resources like **time**, **people**, and **tools**.
 - Set up **monitoring methods** to track how the project is going.
- Even in **agile development**, where plans are more flexible, a **basic startup plan** is still necessary to allocate resources.

3. Development Planning

- As the project progresses, your understanding of the software improves.
 - The **project plan must be updated** regularly to reflect:
 - New insights about the project
 - Changes in **schedule**
 - Revised **cost estimates**
 - Newly identified **risks**
-

Software Pricing

- Software pricing means **estimating the cost** of developing a software system.
- These estimates include:
 - **Hardware costs**
 - **Software tools**
 - **Travel**

- **Training**
- Most importantly, the **effort (work time)** required

⚠ Note: The amount it **costs to develop** the software is **not always the same** as the **price charged to the customer**.

- The final price is also affected by **other business factors**, like:
 - The organization's **goals**
 - **Economic** and **political** situations
 - The **market strategy** of the company

Factors Affecting Software Pricing

Here are some key factors that influence how a software system is priced:

Factor	Explanation
Contractual Terms	If the developer is allowed to keep the source code and reuse it elsewhere, the price can be lower . If the customer demands the full code , the price is higher .
Cost Estimate Uncertainty	If the costs are uncertain , companies may add an extra amount (contingency) to cover unexpected expenses.
Financial Health	If the company is facing financial issues, it may lower the price just to secure the contract and stay in business, even if it means less or no profit .
Market Opportunity	A company may accept a low profit or break-even deal to enter a new market or gain experience for future projects.
Requirements Volatility	If the requirements are expected to change a lot , the company might quote a lower initial price to win the project, and later charge more for changes.

Pricing Strategies

Companies may use different strategies when setting software prices:

1. **Underpricing**

- The company offers the software at a **low price** to:
 - **Keep staff employed** between projects

- **Enter a new market**

2. Increased Pricing

- The price is set **higher** if the client demands a **fixed-price contract**, to **cover possible risks** and changes.

Pricing to Win

- In this strategy, the price is set based on what the **buyer is willing to pay**, not the actual development cost.
 - If the **price is too low**, the company might:
 - Deliver **fewer features** initially
 - **Add features later** for extra charges
 - As **requirements change**, the **new costs** can be **priced higher** to cover earlier losses.
-

Plan-Driven Development

What is Plan-Driven Development?

- **Plan-driven development** is a software engineering approach where the **entire development process is carefully planned** before starting.
- It is based on **traditional engineering project management techniques**.
- It is mainly used for **large software projects**.

How it Works:

- A **project plan** is created at the beginning. This plan includes:
 - What work needs to be done

- Who will do the work
- The schedule/timeline
- What the final outputs (deliverables) will be
- **Project managers** use this plan to:
 - Make decisions
 - Track progress
 - Manage resources and risks

Pros and Cons of Plan-Driven Development

Advantages:

- **Planning early** helps:
 - Consider staff availability and other ongoing projects
 - Spot problems and dependencies **before** they occur

Disadvantages:

- Many early decisions might have to be **changed later** due to:
 - Changing requirements
 - Changes in the development or business environment

What is in a Project Plan?

In a plan-driven approach, the project plan includes:

1. **Introduction** – Overview of the project
2. **Project Organization** – Roles and responsibilities
3. **Risk Analysis** – What could go wrong and how to handle it
4. **Resource Requirements** – Hardware, software, tools needed

5. **Work Breakdown** – Tasks divided into smaller units
6. **Project Schedule** – Timeline for tasks
7. **Monitoring & Reporting** – How progress will be tracked

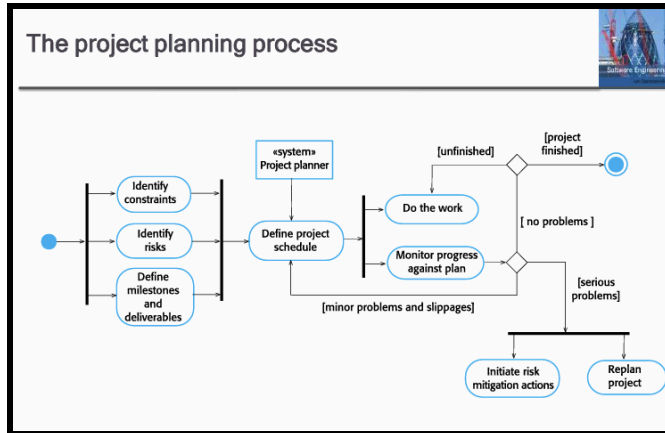
Project Plan Supplements

These are extra plans that support the main project plan:

Plan	What It Describes
Configuration Management	How software versions and changes will be managed
Deployment Plan	How the software and hardware will be delivered to the customer and data migrated
Maintenance Plan	Future maintenance efforts, costs, and resources
Quality Plan	The standards and procedures for maintaining quality
Validation Plan	How the system will be tested and validated

Planning is a Continuous Process

- Planning **starts in the beginning** but **continues throughout** the project.
- You need to **update the plan regularly** based on:
 - New information
 - Team progress
 - Changes in goals or risks



Planning Assumptions

- Don't assume everything will go perfectly.
- Always expect **some problems**, delays, or surprises.
- Make **realistic assumptions** and include **contingency time** in your plan (extra time to handle issues).

Risk Mitigation

- If major problems arise that might **delay the project**, you need to:
 - Take **risk mitigation actions** (steps to reduce the impact)
 - **Replan** the project if needed
 - Possibly **renegotiate** the deadline or project scope with the customer
 - Create a **new schedule** and get the customer's approval

Project Scheduling

Project scheduling is the process of:

- **Breaking down the project** into smaller **tasks**.
- **Deciding when and how** each task will be done.

- **Estimating:**
 - How **long** each task will take (calendar time).
 - How much **effort** it needs (in person-days/months).
 - **Who** will do the task.
 - **Resources** required (e.g., disk space, simulator time, travel budget).

Project Scheduling Activities

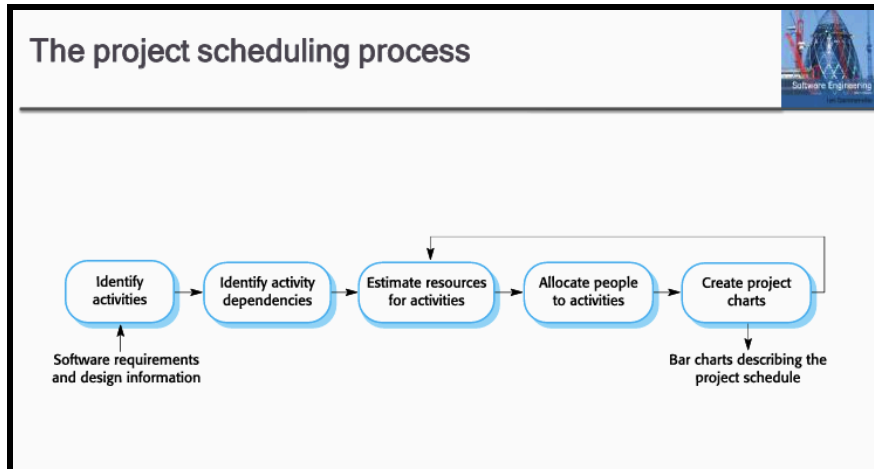
To schedule a project:

1. **Split** the project into tasks.
2. **Estimate time and resources** for each task.
3. **Organize tasks to run in parallel** where possible (to save time).
4. **Reduce dependencies** between tasks to prevent delays.
5. Use **manager's experience** and **intuition** to guide planning.

Scheduling Process & Common Problems

Challenges in scheduling:

- Difficult to accurately estimate task difficulty and cost.
- More people $\neq$ more productivity. Adding people may **slow down** the project (communication overhead).
- **Delays** can happen unexpectedly.
- Always **add contingency** time (extra buffer) for the unexpected.



How is the Schedule Shown?

Two common **graphical formats**:

1. **Bar Charts (Calendar-based)**

- Show tasks and their timelines.
- Easy to understand and track progress.

2. **Activity Networks**

- Show **which tasks depend on others** (dependencies).

 Tip: Don't make tasks too small. A task should take **1–2 weeks**.

Project Activities (Tasks)

Each project task includes:

- **Duration**: Total days or months to finish the task.
- **Effort**: Total person-days/months required.
- **Deadline**: When the task must be done.
- **End-point**: How you know it's done (e.g., a document, test passed, review held).

Milestones vs Deliverables

- **Milestones:** Checkpoints in the project to measure progress (e.g., start of testing).
- **Deliverables:** Items delivered to the client (e.g., design document, working software).

Example Table: Tasks, Durations & Dependencies

Task	Effort (person-days)	Duration (days)	Depends on
T1	15	10	-
T2	8	15	-
T3	20	15	T1 (M1)
T4	5	10	-
T5	5	10	T2, T4 (M3)
T6	10	5	T1, T2 (M4)
T7	25	20	T1 (M1)
T8	75	25	T4 (M2)
T9	10	15	T3, T6 (M5)
T10	20	15	T7, T8 (M6)

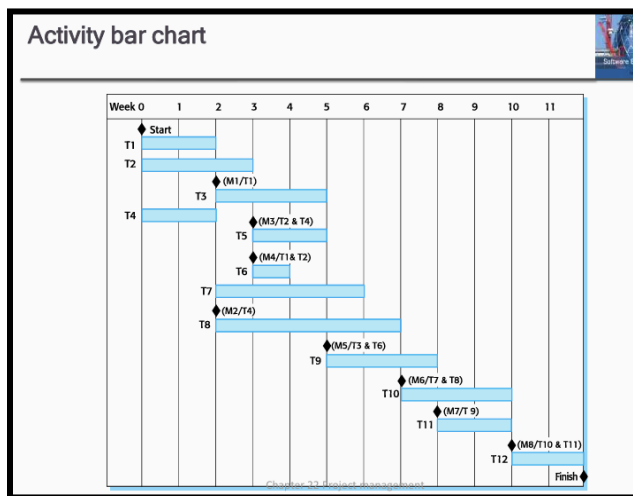
T9 (M7)

T10, T11 (M8)

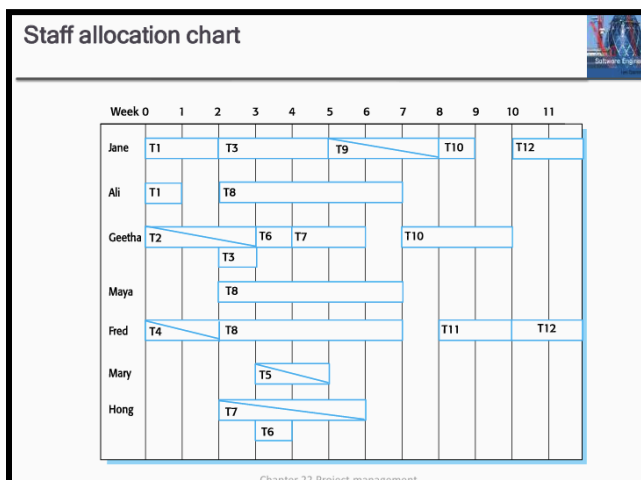
This table helps track **what tasks come first**, how long they take, and **what depends on what**.

Charts for Scheduling

- **Activity Bar Chart:** Shows tasks and time visually like a Gantt chart.



- **Staff Allocation Chart:** Shows how team members are assigned to tasks over time.



Agile Planning

What is Agile Planning?

- Agile planning is used in **Agile methods** of software development, which are **iterative**.
- The software is built and delivered in **increments** (small working parts).
- Unlike **plan-driven** methods, **what features go into each increment is not fixed upfront**.
- Instead, the decision is made **as development happens**, based on:
 - Progress made so far
 - Customer priorities

Since customer needs change over time, Agile uses a **flexible plan** that can **adapt** to these changes.

Stages of Agile Planning

1. **Release Planning**
 - Long-term (covers several months)
 - Decides **which features** will be part of a full **system release**
2. **Iteration Planning**
 - Short-term (usually 2–4 weeks)
 - Focuses on the **next small update** or **increment**

Approaches to Agile Planning

1. Scrum-Based Planning

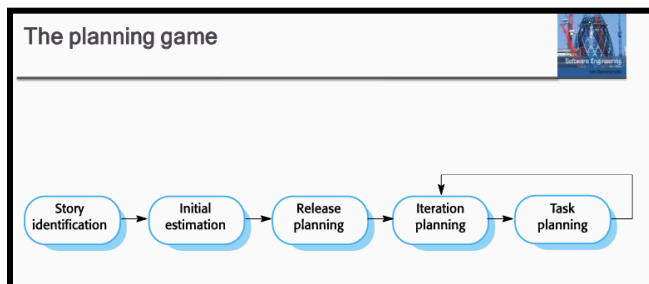
- Uses a **project backlog** (list of pending work)
- Progress and issues are reviewed **daily** in short meetings

2. The Planning Game (from Extreme Programming - XP)

- Based on **user stories** (short descriptions of features)
- Helps estimate effort and track progress

Story-Based Planning (The Planning Game)

- Team reads and discusses each **user story**
- Stories are ranked based on:
 - **Time needed**
 - **Difficulty**
- Each story is given **effort points** (a measure of size/complexity)
- Team's **velocity** is calculated (how many effort points they can finish per day)
- This helps estimate how long the full project will take



Release and Iteration Planning

- In **release planning**:
 - Choose which stories go into a full release
 - Decide the **order** of implementation
- In **iteration planning**:
 - Select stories that can fit within a 2–3 week time frame
 - Use **team's velocity** to avoid overloading

Task Allocation

- Each user story is broken into **development tasks**
 - Each task should take **4–16 hours**
- All tasks for the iteration are listed
- Developers **choose** the tasks they want to do

Benefits:

- Everyone sees the full workload
- Developers feel **ownership** of their tasks, increasing motivation

Software Delivery

- At the end of each iteration, a **working software increment** is delivered
- If tasks take longer than expected:
 - **Features are reduced**
 - **Delivery date stays fixed** — it is **never extended**

Difficulties in Agile Planning

- **Relies heavily on customer involvement**
 - Customers might not always be available
- Some customers may be used to **traditional project plans** and find agile methods **confusing or uncomfortable**

When Agile Planning Works Best

- Works well with:
 - **Small, stable teams**
 - **Teams that can meet regularly** to plan and discuss

- Doesn't work well if:
 - Team is **large or spread across locations**
 - Team members **change frequently**
 - It's hard to arrange **collaborative planning sessions**
-

Estimation Techniques

In software development, it's important for organizations to **estimate the effort and cost** required to complete a project. There are **two main types** of techniques used for this:

1. Experience-Based Techniques

- These depend on the **manager's past experience** with similar projects.
- The manager uses **judgment** to estimate how much effort a project might need.
- Steps usually include:
 - Identifying **deliverables** (the things that must be produced)
 - Listing all **software components or systems**
 - Estimating effort for each item (often in a **spreadsheet**)
 - Adding up all estimates to get the **total effort**
- It's helpful to involve **a group of people** so they can discuss and justify their estimates.

Problem with this method:

- New projects often use **different tools or technologies** (e.g., HTML5, web services).
- If the project is very different from past experience, these estimates might be **inaccurate**.

2. Algorithmic Cost Modeling

- Uses **mathematical formulas** to calculate effort and cost.

- Based on different attributes like:
 - **Product size** (e.g., lines of code)
 - **Process type**
 - **Team experience**

Basic Formula:

ini

CopyEdit

$$\text{Effort} = A \times \text{Size}^B \times M$$

Where:

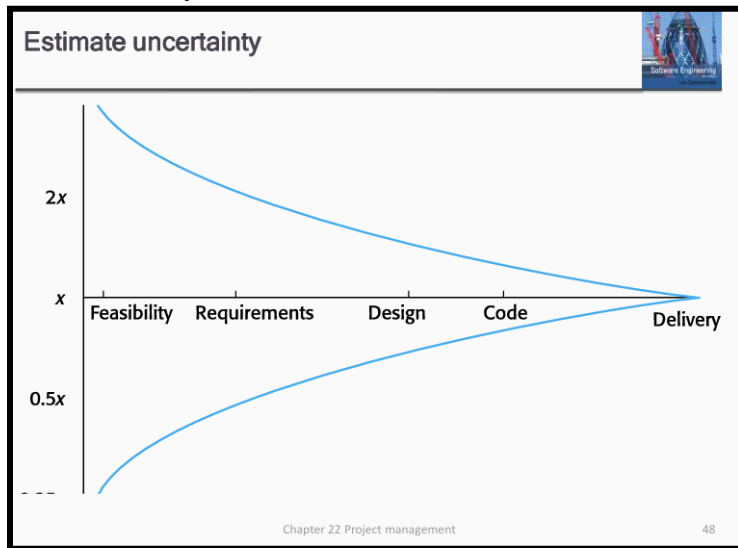
- **A** is a constant based on the organization
- **B** accounts for the **non-linear** effort needed as projects grow
- **M** is a multiplier based on **product, process, and people** factors

✓ Most models use **code size** as the key input, though values for A, B, and M vary.

Uncertainty in Estimates

- You can only know the **actual size** of the software **after** it's built.
- Many factors affect size:
 - Reusing existing components
 - Programming language used
 - Whether the system is distributed (runs on multiple systems)
- As the project **progresses**, size estimates become **more accurate**.

- But factors like **B** and **M** are **subjective** — they rely on the **estimator's judgment**, which adds variability.



Effectiveness of Algorithmic Models

- These models give a **structured way** to estimate effort.
- But they're often **complex and hard to use** in practice.
- Many variables must be estimated, and there's room for **uncertainty**.
- So, they are mostly used by **large organizations**, especially in **defense and aerospace**, where precise estimation is critical.

COCOMO Cost Modeling

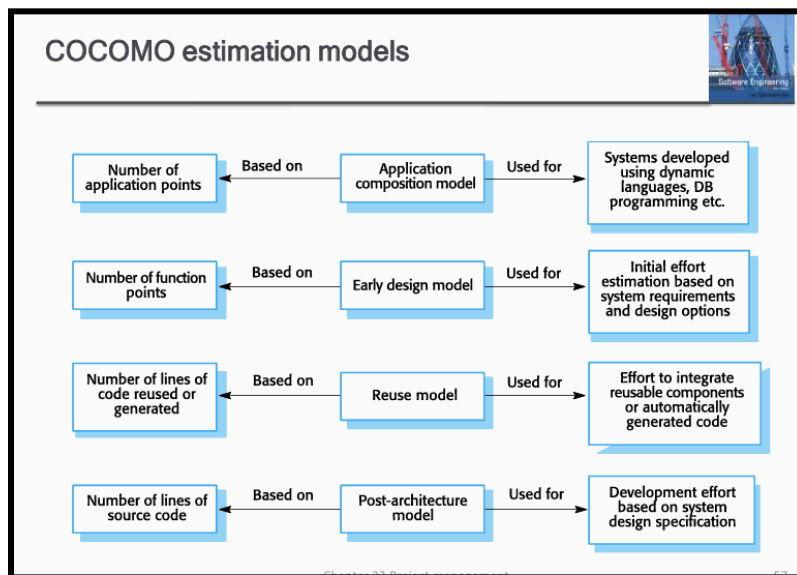
What is COCOMO?

- COCOMO stands for **Constructive Cost Model**.
- It is an **empirical model**, meaning it's based on **real project data** and experience.
- It is an **independent** model, not tied to any specific software vendor.
- The first version came out in **1981 (COCOMO-81)**. It has since evolved into **COCOMO II**, which considers modern development practices like **reuse** and **different methodologies**.

COCOMO II Structure

COCOMO II includes several **sub-models**, each used at different stages of software development:

1. **Application Composition Model** – Used when building software from **existing components**.
2. **Early Design Model** – Used after gathering requirements, before designing.
3. **Reuse Model** – Used when integrating **reusable code** (both modified and unmodified).
4. **Post-Architecture Model** – Used after the architecture is ready and more system details are available.



Application Composition Model

- Useful for **prototyping** or projects with **high reuse**.
- Measures size in **application points** (like screens, reports).

Formula:

ini
CopyEdit

$$PM = (NAP \times (1 - \%reuse/100)) / PROD$$

- Where:

- **PM** = Person-Months (effort)
- **NAP** = Number of Application Points
- **PROD** = Productivity (points/month)

Productivity (PROD) Table:

Developer & Tool Quality	PROD (Points/month)
Very Low	4
Low	7
Nominal	13
High	25
Very High	50

Early Design Model

- Used when **requirements** are available but **design** hasn't started.

Formula:

ini
CopyEdit
 $PM = A \times \text{Size}^B \times M$

- - **A** = **2.94** (default)
 - **Size** = in **KLOC** (thousands of lines of code)
 - **B** = 1.1 to 1.24, based on factors like project risk and novelty

M = product of **7 multipliers**:

ini
CopyEdit
 $M = \text{PERS} \times \text{RCPX} \times \text{RUSE} \times \text{PDIF} \times \text{PREX} \times \text{FCIL} \times \text{SCED}$

○

- **Multipliers explained:**

- **RCPX** = Product reliability and complexity
- **RUSE** = Amount of reuse required
- **PDIF** = Difficulty of the platform
- **PREX** = Experience of personnel
- **PERS** = Capability of personnel
- **SCED** = Scheduling constraints
- **FCIL** = Quality of facilities

Reuse Model

Used when reusing existing code:

1. Black-box Reuse (no changes to code)

ini

CopyEdit

$$PM = (ASLOC \times AT/100) / ATPROD$$

- - **ASLOC** = Adapted Source Lines of Code
 - **AT** = % of code auto-generated
 - **ATPROD** = Productivity during integration

2. White-box Reuse (modifying reused code)

ini

CopyEdit

$$ESLOC = ASLOC \times (1 - AT/100) \times AAM$$

- - **ESLOC** = Equivalent new lines of code
 - **AAM** = Adaptation Adjustment Multiplier (based on effort needed to change, understand, and decide on reused code)

Post-Architecture Model

Same formula as Early Design Model:

ini

CopyEdit

$$PM = A \times \text{Size}^B \times M$$

- - But uses **17 multipliers** instead of 7.
 - **Size** includes:
 - New code
 - Equivalent new code from reuse model
 - Code modified due to **requirement changes**

The Exponent "B"

- The exponent **B** affects how the size impacts the overall effort.

It's calculated as:

ini

CopyEdit

$$B = 1.01 + (\text{Sum of 5 scale factors}) / 100$$

●

Scale Factors:

Factor	Meaning
Precedentedness	Is this type of project new to the company?
Development Flexibility	How flexible is the process?
Architecture/Risk Resolution	Has risk analysis been done?
Team Cohesion	How well does the team work together?
Process Maturity	How mature is the organization's process? (based on CMM rating)

Example:

If scale factor total = 16,

ini

CopyEdit

$$B = 1.01 + 16/100 = 1.17$$

♦ Types of Multipliers

These affect the **M** value in the formula:

1. **Product Attributes** – e.g., software reliability or complexity
2. **Computer Attributes** – constraints from hardware or platform
3. **Personnel Attributes** – team's skills and experience
4. **Project Attributes** – things like schedule, tools, and facilities

Effect of Cost Drivers – Example

Let's say:

- **System Size:** 128,000 DSI (Delivered Source Instructions)
- **Exponent B:** 1.17
- **Initial Estimate:** 730 person-months

Now, with **high multipliers** like:

- Reliability = 1.39
- Complexity = 1.30
- Memory = 1.21
- Tool Use = 1.12
- Schedule = 1.29

Adjusted Effort = 2,306 person-months

In contrast, with **low multipliers**:

- Reliability = 0.75
- Complexity = 0.75
- Tool Use = 0.72

Adjusted Effort = 295 person-months

This shows how **cost drivers greatly affect** the final effort estimate.

Project Duration & Staffing

- Besides effort, we also estimate the **calendar time** (duration).

Formula:

ini
CopyEdit
$$TDEV = 3 \times (PM)^{(0.33 + 0.2 \times (B - 1.01))}$$

- - **TDEV** = time in months
 - **PM** = person-months (effort)
 - **B** = exponent (as above)
 - Note: **Duration doesn't directly depend** on how many people are on the project.
 - **You can't divide total work by schedule** to get people needed.
 - The number of people varies by project phase.
 - Adding too many people too quickly can lead to **delays**, not faster delivery.
-